

How the Template Engine Works

Data in → template chosen → slots filled → screen out. One engine, 26 workflows, 105 screens.

5

templates
(whole engine)

50

governed slots
(from 1,137 raw)

19

reusable
components

26

workflows
covered

0

new code for
workflow #27

1 The flow — one real screen, end to end

Example: **S9.5 Competitive Repricing** → **Decide stage**. Same 5 steps run for all 105 screens.

1

Backend sends data

One "Decision Object" per screen.
Pure business facts — no layout,
no CSS.

```
{ story_code: "S9.5", stage: "decide",  
  action_type: "reprice", cardinality: "many",  
  proposal: { slate: [6 items], ... }, guardrails: {...} }
```

2

selectTemplate()

Reads just 4 fields. A lookup table
— no AI, no scoring, no network
call.

```
// tries 3 keys, most specific first  
"reprice|decide|many" → miss  
"*|decide|many" → miss  
"*|decide|*" → HIT
```

3

Template resolved

A JSON layout spec. Says which
slots exist, where they sit, which
buttons show.

```
decide.slate.v1  
17 slots · 6 sections · 6 actions  
sections: recommendation, slate | guardrails, totals, context, basis
```

4

Slots bind to data

Each slot = a fixed data path + a
fixed component. This is the
governed vocabulary.

```
slate: { binding: 'proposal.slate', blockType: 'table' }  
guardrail_verdict: { binding: 'guardrails.verdict', ... }  
// data missing → slot omitted, not shown empty
```

5

Blocks render

Shape-validated, each in its own
error boundary. One bad card can't
kill the page.

```
table → <TableBlock> (6 rows, selectable)  
statList → <StatListBlock>  
→ Live screen + Approve / Modify / Dismiss
```

The one design choice that matters: the backend **never** sends a template ID. It sends business facts; the frontend decides layout. So a screen redesign is a frontend deploy — not a cross-team backend change.

2 Why this is cheap to grow

WHAT CHANGES	COST
New workflow (#27, #28...)	2 data files. Zero new components, zero engine change.
New layout for one stage	1 row in the lookup table + 1 template JSON.
New data field on screen	1 slot entry (path + component). No renderer change.
Brand-new component type (rare)	1 component + 1 registry line + 1 validator.

Questions the CTO will likely ask

Straight answers — including what isn't done yet.

3 Architecture questions

"Why 5 templates? Why not one per workflow?"

Because layout is driven by

stage

, not by workflow. Every Decide screen is the same shape — a recommendation, a slate of items, guardrails. The old approach had 105 hand-built manifests with 835 different slot names, 667 used exactly once. Now it's 50 governed slots, enforced by a test so it can't regress.

"What happens when a proposal doesn't match any template?"

It fails loudly — an explicit "can't display this" state. The selector

never

falls back to a default layout, because silently rendering the wrong screen is worse than rendering nothing: the operator can't tell it happened.

"What if a field is missing or malformed?"

Three layers. The API response is contract-validated before the app sees it. A slot whose data is absent is

omitted

, not shown empty. And every block sits in its own error boundary, so one bad card renders a small placeholder instead of white-screening the page.

"How fast is template selection?"

O(1) — three object-key lookups, synchronous, in-memory. No network call, no model, no scoring engine.

4 The "is it production-ready?" questions

"Is it connected to a real backend?"

No.

It runs on an in-memory test double serving the same contract a real API must implement. The swap-in is one env var (VITE_API_BASE_URL) — the HTTP client, retry policy, idempotency keys and error taxonomy are already built and tested.

"Is there auth?"

No — zero.

No token, no session, no 401 handling anywhere in the codebase. Anyone with a link opens any screen. This is the hard blocker before any rollout.

"Do all the action buttons work?"

Six operator actions are defined (approve, approve-selected, modify, send back, snooze, dismiss) and all run through one shared rule function used by the UI, the dispatch layer and the mock server. But the sample data makes snooze permanently ineligible — no deadlines in the dataset — so it's a dead button today.

"Any known bugs?"

Yes, one that matters and I'd flag proactively: on 16 of 26 Decide screens the slate rows have no stable ID, so

selecting a row can approve the wrong row

. Fix is for the backend to send an `item_id` on every row — positional IDs don't survive sorting or pagination. Also the 627 KB sample dataset currently ships inside the production bundle (~78% of it).

"How much of the data is real?"

~39% of reference fields carry through to the screens. The rest is presentation junk the pipeline is *supposed* to strip. Stronger guarantee than the percentage: every field that renders comes from the source field that actually means it — nothing is shape-guessed or substituted.

If you only say one thing: "The hard, reusable part is done and tested — one engine instead of 105 screens. What's left is integration: backend, auth, and stable item IDs. None of that needs more frontend design."